

Deploying WordPress on Docker with NFS & Separate Networks

Objective Recap

- ✓ Deploy WordPress on Docker
 - ✓ Use NFS for High Data Availability
 - ✓ Ensure Data Persistence with Docker Volumes
 - ✓ Isolate WordPress and MySQL Containers with Separate Networks
-

Prerequisites

- Basic knowledge of Docker
 - Docker installed on your machine
 - AWS EC2 instances ready:
 - **VM 1:** nfs-server.com (NFS Server)
 - **VM 2:** docker.com (Docker Host)
-

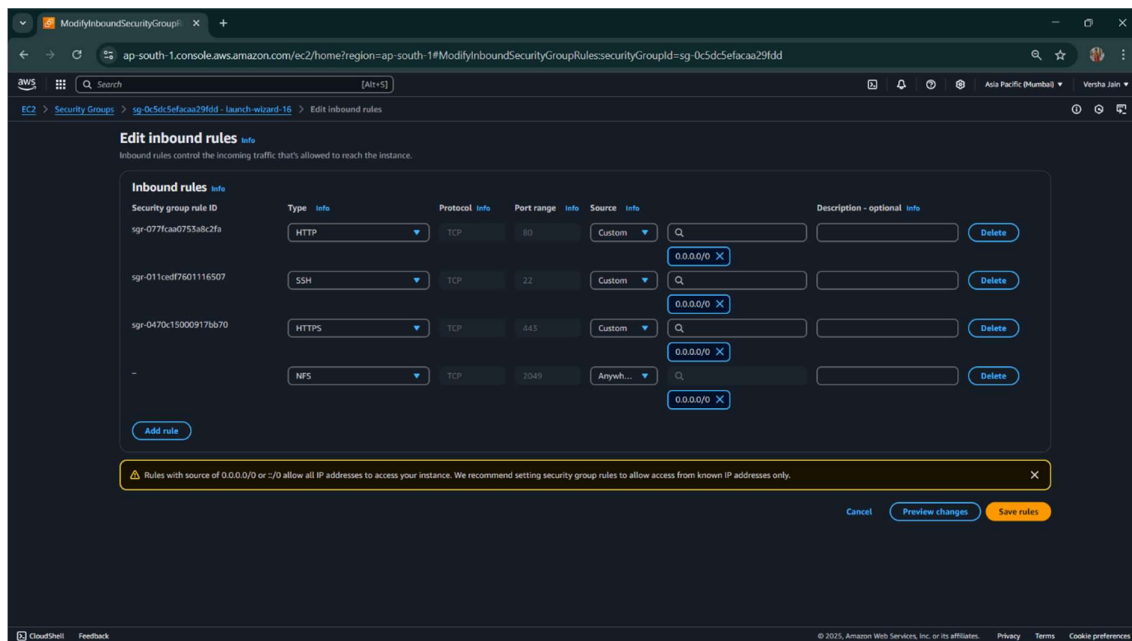
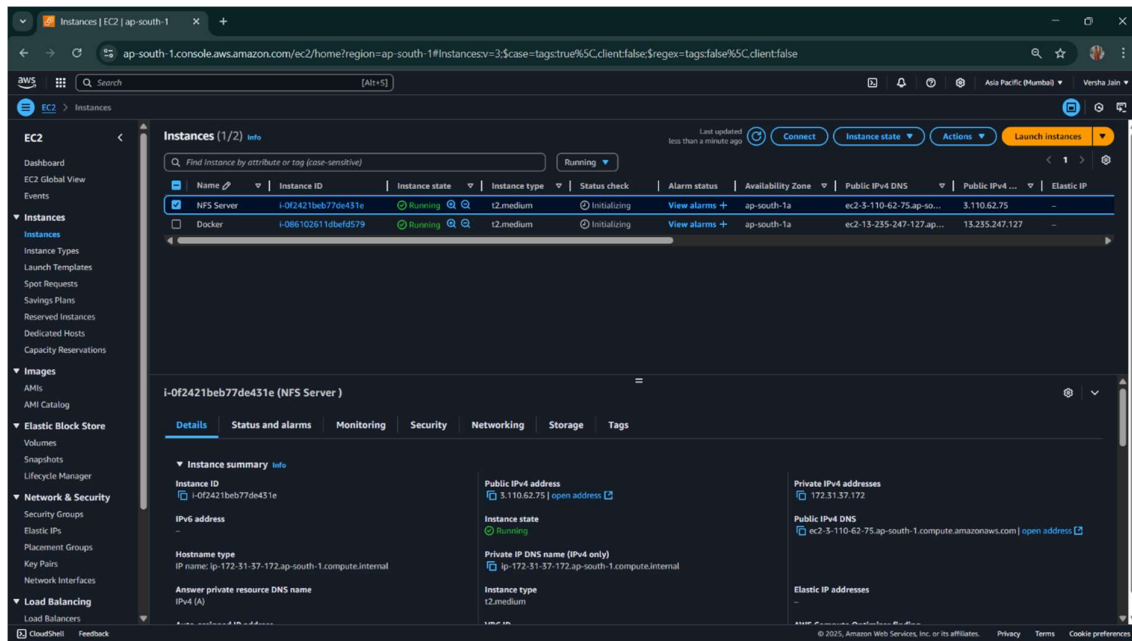
Step-by-Step Guide

Step 1: Create EC2 Instances

1. **VM 1:** nfs-server.com (Amazon Linux/Ubuntu)
2. **VM 2:** docker.com (Amazon Linux/Ubuntu)

✓ Security Group Configuration:

- VM 1 (nfs-server.com):
 - 2049/tcp → NFS communication
- VM 2 (docker.com):
 - 80/tcp → HTTP access for WordPress
 - 2049/tcp → NFS communication



Step 2: Configure NFS Server on nfs-server.com

Step 1: Install NFS Utilities

Amazon Linux / RHEL

```
sudo yum update -y
```

```
sudo yum install -y nfs-utils
```

```
root@ip-172-31-37-172:~  
[root@nfsserver ~]# sudo systemctl enable --now nfs-server  
Created symlink /etc/systemd/system/multi-user.target.wants/nfs-server.service → /usr/lib/systemd/system/nfs-server.service.  
[root@nfsserver ~]# sudo mkdir -p /wp_data  
[root@nfsserver ~]# sudo mkdir -p /db_data  
[root@nfsserver ~]# sudo chmod 777 /wp_data /db_data  
[root@nfsserver ~]# ls /  
afs bin boot dev efi etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var wp_data  
[root@nfsserver ~]# sudo vi /etc/exports  
[root@nfsserver ~]# cat /etc/exports  
/wp_data *(rw,sync,no_root_squash)  
/db_data *(rw,sync,no_root_squash)  
[root@nfsserver ~]# |
```

Step 2: Start and Enable NFS

sudo systemctl enable --now nfs-server

```
root@ip-172-31-37-172:~  
[root@nfsserver ~]# sudo systemctl enable --now nfs-server  
Created symlink /etc/systemd/system/multi-user.target.wants/nfs-server.service → /usr/lib/systemd/system/nfs-server.service.  
[root@nfsserver ~]# sudo mkdir -p /wp_data  
[root@nfsserver ~]# sudo mkdir -p /db_data  
[root@nfsserver ~]# sudo chmod 777 /wp_data /db_data  
[root@nfsserver ~]# ls /  
afs bin boot dev efi etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var wp_data  
[root@nfsserver ~]# sudo vi /etc/exports  
[root@nfsserver ~]# cat /etc/exports  
/wp_data *(rw,sync,no_root_squash)  
/db_data *(rw,sync,no_root_squash)  
[root@nfsserver ~]# sudo exportfs -arv  
exporting */db_data  
exporting */wp_data  
[root@nfsserver ~]# sudo systemctl restart nfs-server  
[root@nfsserver ~]# |
```

Step 3: Create Directories for WordPress & MySQL

sudo mkdir -p /wp_data

sudo mkdir -p /db_data

Set permissions

sudo chmod 777 /wp_data /db_data

```
root@ip-172-31-37-172:~  
[root@nfsserver ~]# sudo systemctl enable --now nfs-server  
Created symlink /etc/systemd/system/multi-user.target.wants/nfs-server.service → /usr/lib/systemd/system/nfs-server.service.  
[root@nfsserver ~]# sudo mkdir -p /wp_data  
[root@nfsserver ~]# sudo mkdir -p /db_data  
[root@nfsserver ~]# sudo chmod 777 /wp_data /db_data  
[root@nfsserver ~]# ls /  
afs bin boot dev efi etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var wp_data  
[root@nfsserver ~]# sudo vi /etc/exports  
[root@nfsserver ~]# cat /etc/exports  
/wp_data *(rw,sync,no_root_squash)  
/db_data *(rw,sync,no_root_squash)  
[root@nfsserver ~]# sudo exportfs -arv  
exporting */db_data  
exporting */wp_data  
[root@nfsserver ~]# sudo systemctl restart nfs-server  
[root@nfsserver ~]# |
```

Step 4: Configure NFS Export Rules

sudo nano /etc/exports

Add the following:

/wp_data docker.com(rw,sync,no_root_squash)

/db_data docker.com(rw,sync,no_root_squash)

Step 5: Export the Configuration and Restart NFS

`sudo exportfs -arv`

`sudo systemctl restart nfs-server`

✅ NFS Server is Ready!

```
root@ip-172-31-37-172:~  
[root@nfsserver ~]# sudo systemctl enable --now nfs-server  
Created symlink /etc/systemd/system/multi-user.target.wants/nfs-server.service → /usr/lib/systemd/system/nfs-server.service.  
[root@nfsserver ~]# sudo mkdir -p /wp_data  
[root@nfsserver ~]# sudo mkdir -p /db_data  
[root@nfsserver ~]# sudo chmod 777 /wp_data /db_data  
[root@nfsserver ~]# ls /  
afs bin boot dev efi etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var wp_data  
[root@nfsserver ~]# sudo vi /etc/exports  
[root@nfsserver ~]# cat /etc/exports  
/wp_data *(rw,sync,no_root_squash)  
/db_data *(rw,sync,no_root_squash)  
[root@nfsserver ~]# sudo exportfs -arv  
exporting *:db_data  
exporting *:wp_data  
[root@nfsserver ~]# sudo systemctl restart nfs-server  
[root@nfsserver ~]#
```

🐳 Step 3: Configure Docker Host on docker.com

Step 1: Install Docker

Amazon Linux / RHEL

`sudo yum update -y`

`sudo yum install -y docker`

Set up the repository

Install the `dnf-plugins-core` package (which provides the commands to manage your DNF repositories) and set up the repository.

```
$ sudo dnf -y install dnf-plugins-core  
$ sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
```

Install Docker Engine

1. Install the Docker packages.

Latest Specific version

To install the latest version, run:

```
$ sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

If prompted to accept the GPG key, verify that the fingerprint matches

`060A 61C5 1B55 8A7F 742B 77AA C52F EB6B 621E 9F35`, and if so, accept it.

This command installs Docker, but it doesn't start Docker. It also creates a `docker` group, however, it doesn't add any users to the group by default.

2. Start Docker Engine.

```
$ sudo systemctl enable --now docker
```

Step 2: Start and Enable Docker

sudo systemctl enable --now docker

Step 3: Create Directories to Mount NFS Shares

sudo mkdir -p /mnt/wp_data

sudo mkdir -p /mnt/db_data

```
root@ip-172-31-37-172:~ x root@ip-172-31-38-238:~ x + v
[root@docke ~]# sudo mkdir -p /mnt/wp_data
[root@docke ~]# sudo mkdir -p /mnt/db_data
[root@docke ~]# ls /mnt/
db_data wp_data
[root@docke ~]# |
```

Step 4: Mount NFS Shares

sudo mount -t nfs nfs-server.com:/wp_data /mnt/wp_data

sudo mount -t nfs nfs-server.com:/db_data /mnt/db_data

Step 5: Make Mounts Persistent

Add these lines in /etc/fstab:

nfs-server.com:/wp_data /mnt/wp_data nfs defaults 0 0

nfs-server.com:/db_data /mnt/db_data nfs defaults 0 0

```
root@ip-172-31-37-172:~ x root@ip-172-31-38-238:~ x + v
UUID=3574dc3c-6767-4d8a-989a-c853621b641c / xfs defaults 0 0
UUID=5d2bec95-386f-4310-8797-8c1d4a393e12 /boot xfs defaults 0 0
UUID=7B77-95E7 /boot/efi vfat defaults,uid=0,gid=0,umask=077,shortname=winnt 0 2
3.110.62.75:/wp_data /mnt/wp_data nfs defaults 0 0
3.110.62.75:/db_data /mnt/db_data nfs defaults 0 0
~
~
~
```

🌐 Step 4: Create Docker Networks

docker network create wp_network

docker network create db_network

```
root@ip-172-31-37-172:~ x root@ip-172-31-38-238:~ x + v
[root@docke ~]# docker network create wp_network
bf0947d72e6ebdd77f489508c706e7710d76824a92d7779b3a55fdfb6e480691
[root@docke ~]# docker network create db_network
50ac110a4fb50412d62923fa3563aee5c134bbe5fc6d6009867073373a1a27a7
[root@docke ~]# docker network ls
NETWORK ID NAME DRIVER SCOPE
7ab9b96b5893 bridge bridge local
50ac110a4fb5 db_network bridge local
f46a47a22cd3 host host local
d03632892e96 none null local
bf0947d72e6e wp_network bridge local
[root@docke ~]# |
```

Step 5: Pull Docker Images

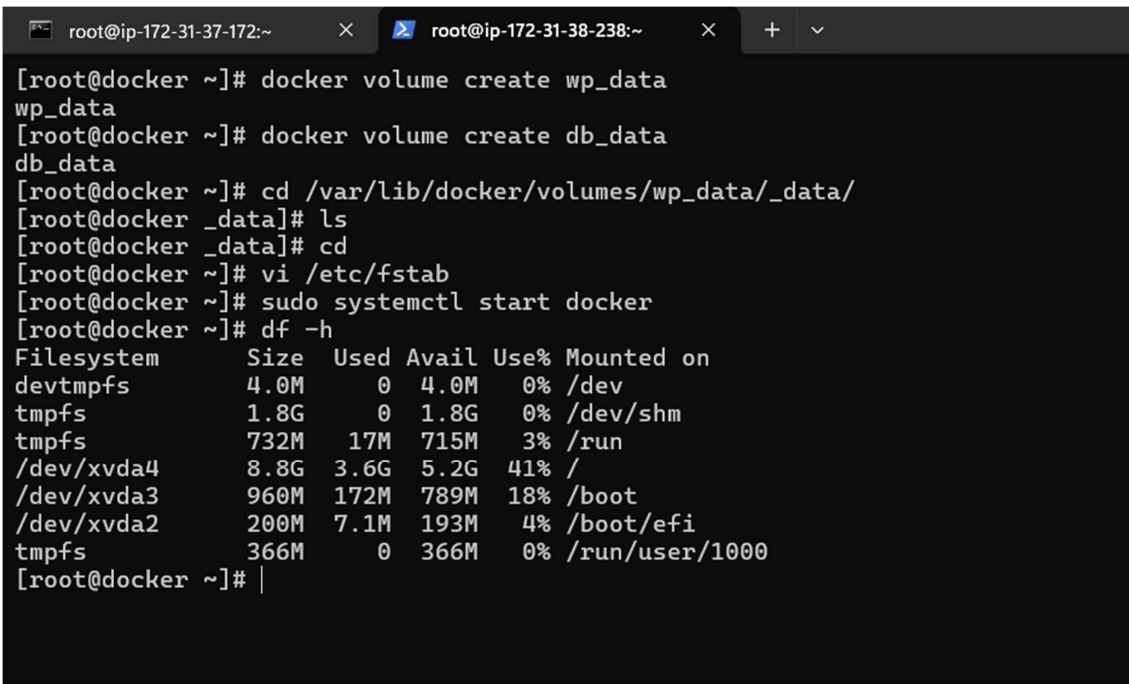
```
docker pull mysql:5.7
```

```
docker pull wordpress:latest
```

Step 6: Create Docker Volumes with NFS

```
docker volume create --name wp_vol --opt type=nfs --opt o=addr=nfs-server.com,rw --opt device=:/wp_data
```

```
docker volume create --name db_vol --opt type=nfs --opt o=addr=nfs-server.com,rw --opt device=:/db_data
```



```
root@ip-172-31-37-172:~ x root@ip-172-31-38-238:~ x + v
[root@docker ~]# docker volume create wp_data
wp_data
[root@docker ~]# docker volume create db_data
db_data
[root@docker ~]# cd /var/lib/docker/volumes/wp_data/_data/
[root@docker _data]# ls
[root@docker _data]# cd
[root@docker ~]# vi /etc/fstab
[root@docker ~]# sudo systemctl start docker
[root@docker ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0    4.0M   0% /dev
tmpfs           1.8G   0    1.8G   0% /dev/shm
tmpfs           732M  17M   715M   3% /run
/dev/xvda4       8.8G  3.6G   5.2G  41% /
/dev/xvda3       960M  172M   789M  18% /boot
/dev/xvda2       200M   7.1M   193M   4% /boot/efi
tmpfs           366M   0    366M   0% /run/user/1000
[root@docker ~]# |
```

Step 7: Run MySQL Container

```
docker run -d \
  --name mysql \
  --network db_network \
  -e MYSQL_ROOT_PASSWORD=redhat \
  -v db_vol:/var/lib/mysql \
  mysql:5.7
```

 **MySQL is running now!**

```
root@ip-172-31-37-172 ~ x root@ip-172-31-39-238 ~ x + v
[root@docker ~]# docker volume create db_data
db_data
[root@docker ~]# cd /var/lib/docker/volumes/wp_data/_data/
[root@docker _data]# ls
[root@docker _data]# cd
[root@docker ~]# vi /etc/fstab
[root@docker ~]# sudo systemctl start docker
[root@docker ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M  0  4.0M   0% /dev
tmpfs           1.8G  0  1.8G   0% /dev/shm
tmpfs           732M  17M  715M   3% /run
/dev/xvda4      8.8G  3.6G  5.2G  41% /
/dev/xvda3      960M  172M  789M  18% /boot
/dev/xvda2      200M  7.1M  193M   4% /boot/efi
tmpfs          366M  0  366M   0% /run/user/1000
[root@docker ~]# docker run -d \
  --name wordpress \
  --restart always \
  -v wp_vol:/var/www/html \
  -p 80:80 \
  wordpress
ea64c863c1d83325d356c86e74f7a95f3d1985966d84553e3a4f6603f56f1cfc
[root@docker ~]# docker run -d --name db -v db_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=redhat@00 mysql:latest
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
804bb8ae89de: Pull complete
1b515e7ceb69: Pull complete
ea11c8a9f08: Pull complete
8d18181893b8: Pull complete
e8a910cc8608: Pull complete
bc0c792ca096: Pull complete
8d73d2a73425: Pull complete
4a7e00d873b9: Pull complete
27a2553d6a80: Pull complete
69e76254f502: Pull complete
Digest: sha256:9b9d0aa4860798acff13d2a8e3bc26639fe18b83fa5cd3e3d8e16b3ed05dd
Status: Downloaded newer image for mysql:latest
1439aa81e7bc45070e17ddac31c18d478de57f8df27847bf60a0d32262365
[root@docker ~]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
1439aa81e7bc   mysql:latest   "docker-entrypoint.s..." 8 seconds ago  Up 7 seconds  3306/tcp, 33060/tcp                db
ea64c863c1d8   wordpress     "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp  wordpress
596291262f3e   wordpress     "docker-entrypoint.s..." 7 minutes ago  Exited (0) 5 minutes ago                                wordpress_debug
```

Step 8: Configure MySQL for WordPress

Step 1: Access MySQL Container

```
docker exec -it mysql mysql -u root -p
```

Enter password: redhat

Step 2: Create Database and User

```
CREATE DATABASE wp_db;
```

```
CREATE USER 'wp_user'@'%' IDENTIFIED BY 'redhat';
```

```
GRANT ALL PRIVILEGES ON wp_db.* TO 'wp_user'@'%';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

```
root@ip-172-31-37-172 ~ x root@ip-172-31-39-238 ~ x + v
[root@docker ~]# docker exec -it db bash
bash-5.1# ls
afs bin boot dev docker-entrypoint-initdb.d etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 0
Server version: 9.2.0 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database wordpress_db;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE USER 'wp_user'@'%' IDENTIFIED BY 'redhat';
Query OK, 0 rows affected (0.03 sec)

mysql> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'%';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> exit
Bye
bash-5.1# |
```

🔗 Step 9: Run WordPress Container

```
docker run -d \
  --name wordpress \
  --network wp_network \
  -p 80:80 \
  -v wp_vol:/var/www/html \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_USER=wp_user \
  -e WORDPRESS_DB_PASSWORD=redhat \
  -e WORDPRESS_DB_NAME=wp_db \
  wordpress:latest
```

✅ WordPress is running now!

```
root@ip-172-31-37-172:~ root@ip-172-31-38-238:~
[root@docker ~]# docker volume create db_data
db_data
[root@docker ~]# cd /var/lib/docker/volumes/wp_data/_data/
[root@docker _data]# ls
[root@docker _data]# cd
[root@docker ~]# vi /etc/fstab
[root@docker ~]# sudo systemctl start docker
[root@docker ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M  0  4.0M   0% /dev
tmpfs           1.8G  0  1.8G   0% /dev/shm
tmpfs           732M  17M  715M   3% /run
/dev/xvda4      8.8G  3.6G  5.2G  41% /
/dev/xvda3      960M  172M  789M  18% /boot
/dev/xvda2      200M   7.1M  193M   4% /boot/efi
tmpfs          366M  0  366M   0% /run/user/1000
[root@docker ~]# docker run -d \
  --name wordpress \
  --restart always \
  -v wp_vol:/var/www/html \
  -p 80:80 \
  wordpress
ea64c863c1d833254356c86e74f7a05f3d1905966d84553e3a4f6603f56f1cfc
[root@docker ~]# docker run -d --name db -v db_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=redhat@00 mysql:latest
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
804bb8ae89de: Pull complete
1b515e7ceb69: Pull complete
ea11c8a9f08: Pull complete
8d181893b8: Pull complete
e0a910cc8604: Pull complete
bc0c792ca096: Pull complete
8d73d2a73425: Pull complete
4a7e00d873b9: Pull complete
27a2553d6a80: Pull complete
69e76254f502: Pull complete
Digest: sha256:9b9d0aab4860798acff13d2a0ece3bc26639fe18b83fa5cd3e3d0e16b3ed05dd
Status: Downloaded newer image for mysql:latest
1439aa01e7bc7bc450970e17ddac31c18d470de57f0df27847bf60a0d32262365
[root@docker ~]# docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
1439aa01e7bc   mysql:late "docker-entrypoint.s..." 8 seconds ago Up 7 seconds  3306/tcp, 33060/tcp                db
ea64c863c1d8   wordpress  "docker-entrypoint.s..." 2 minutes ago Up 2 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp  wordpress
596291262f3e   wordpress  "docker-entrypoint.s..." 7 minutes ago Exited (0) 5 minutes ago      wordpress_debug
```

🔥 Step 10: Configure wp-config.php

Step 1: Enter WordPress Container

```
docker exec -it wordpress bash
```

Step 2: Copy wp-config-sample.php

```
cd /var/www/html
```


cp wp-config-sample.php wp-config.php

Step 3: Edit wp-config.php

nano wp-config.php

Update the following fields:

define('DB_NAME', 'wp_db');

define('DB_USER', 'wp_user');

define('DB_PASSWORD', 'redhat');

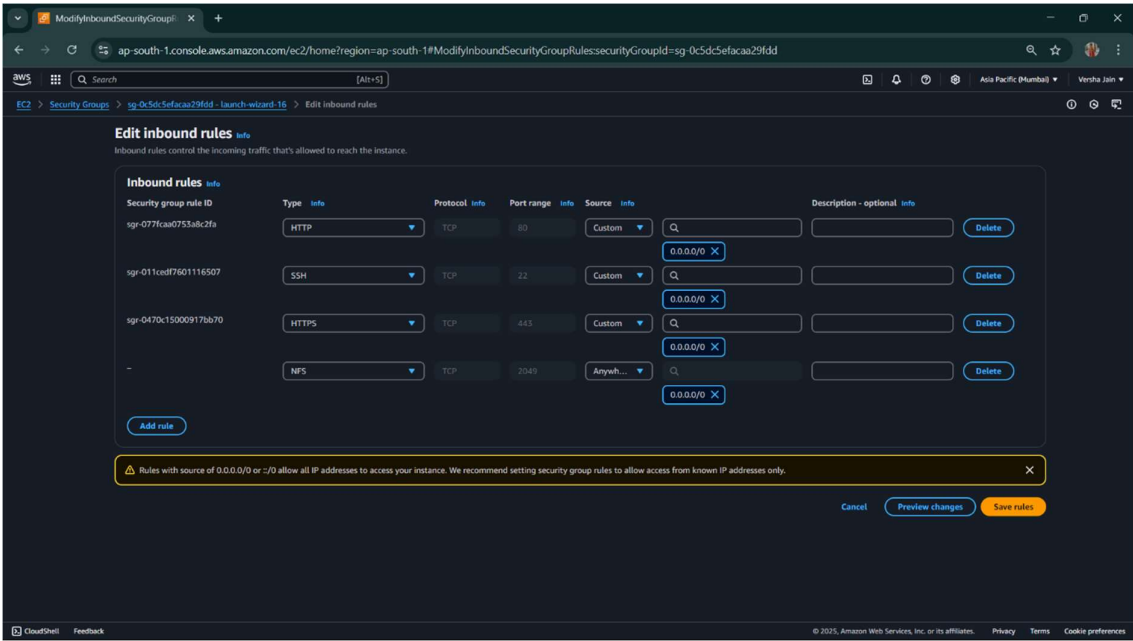
define('DB_HOST', 'mysql');

Save and exit!

```
Last login: Wed Mar 26 12:23:53 2025 from 49.43.178.183
[ec2-user@nfsserver ~]$ sudo -i
[root@nfsserver ~]# ls
[root@nfsserver ~]# ls /
afs bin boot dev efi etc home lib lib64 media mnt opt proc root run sbin srv sys top usr var wp_data
[root@nfsserver ~]# cd /wp_data/
[root@nfsserver wp_data]# ls
index.php wp-activate.php wp-comments-post.php wp-content wp-links-opsml.php wp-mail.php wp-trackback.php
license.txt wp-admin wp-config-docker.php wp-cron.php wp-load.php wp-settings.php xmlrpc.php
readme.html wp-blog-header.php wp-config-sample.php wp-includes wp-login.php wp-signup.php
[root@nfsserver wp_data]#
```

Step 11: Open Security Groups

- Open 80/tcp in Docker Host’s security group to allow HTTP traffic.
- Open 2049/tcp in both NFS and Docker host for NFS communication.

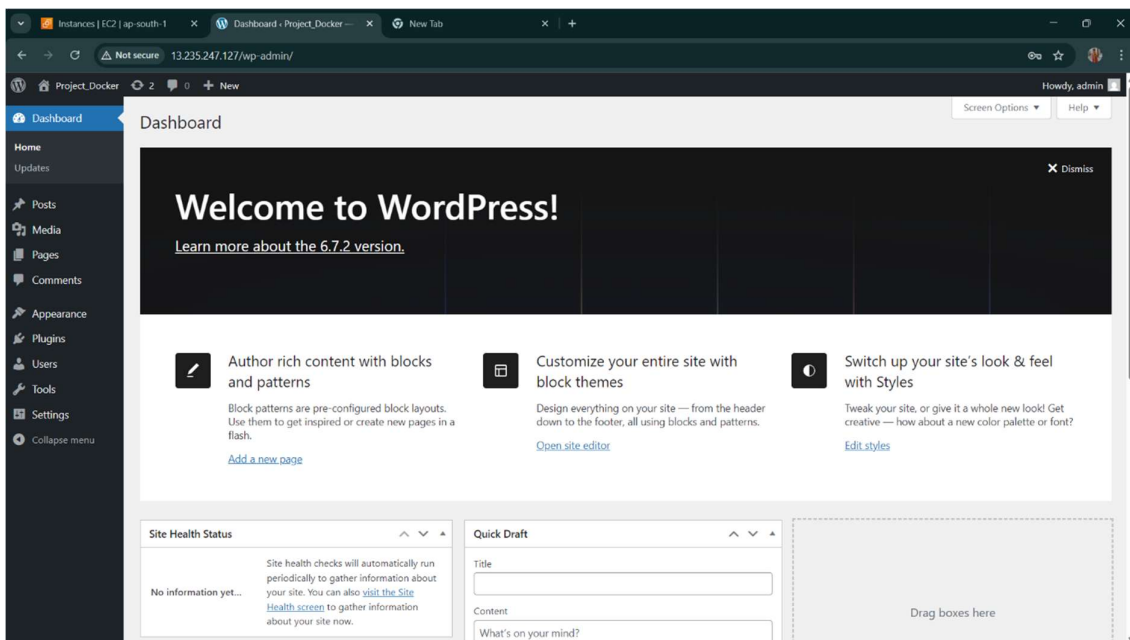
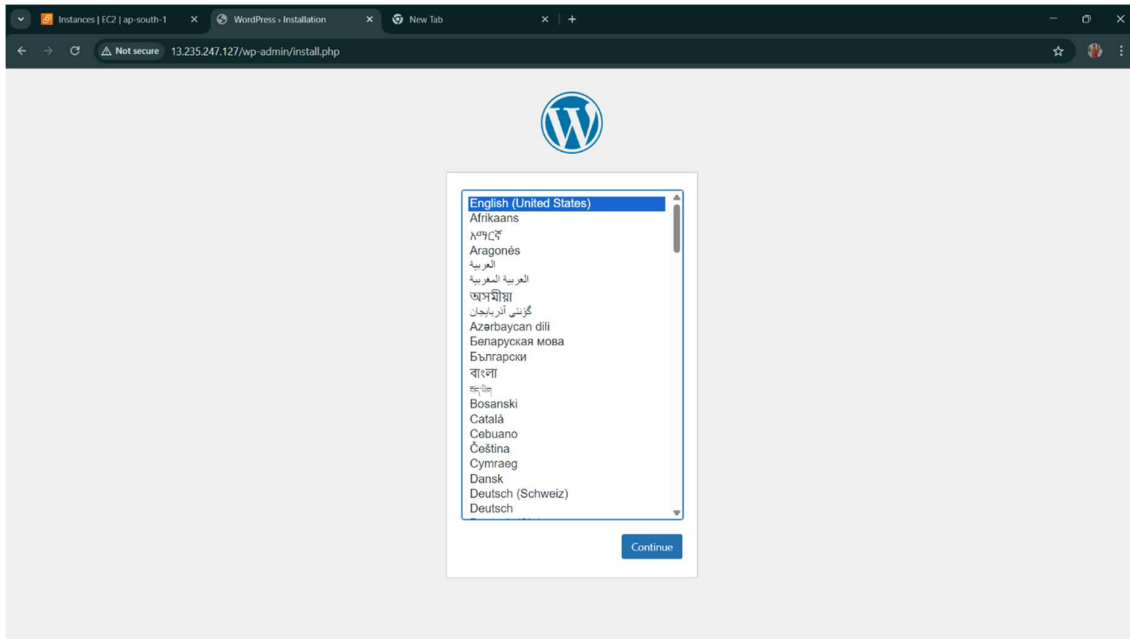


Step 12: Access WordPress!

Open your browser and navigate to:

`http://<docker-host-public-IP>`

✓ You should see the WordPress installation screen. Follow the prompts to complete the installation!



Mounted NFS Directories

-> In Instance 1 (NFS-Server)

```
Last login: Wed Mar 26 12:23:53 2025 from 49.43.178.183
[ec2-user@nfsserver ~]$ sudo -i
[root@nfsserver ~]# ls
[root@nfsserver ~]# ls /
efs bin boot dev efi etc home lib lib64 media mnt opt proc root run sbin srv sys top usr var wp_data
[root@nfsserver ~]# cd /wp_data/
[root@nfsserver wp_data]# ls
index.php wp-activate.php wp-comments-post.php wp-content wp-links-opml.php wp-mail.php wp-trackback.php
license.txt wp-admin wp-config-docker.php wp-cron.php wp-load.php wp-settings.php xmlrpc.php
readme.html wp-blog-header.php wp-config-sample.php wp-includes wp-login.php wp-signup.php
[root@nfsserver wp_data]#
```

-> In Instance 2 (Docker)

```
root@nfsserver:~ root@ip-172-31-38-238:/var/lib/docker/volumes/
[root@docker wp_vol]# cd /var/lib/docker/volumes/
[root@docker volumes]# ls
backingFsBlockDev c99abb8d97ea8dc4655a7a9df6cdca57e6cfff9b6940bdf3e6fcaa2c25a340b74 db_data metadata.db wp_data wp_vol
[root@docker volumes]# cd wp_vol/
[root@docker wp_vol]# ls
index.php wp-activate.php wp-comments-post.php wp-content wp-links-opml.php wp-mail.php wp-trackback.php
license.txt wp-admin wp-config-docker.php wp-cron.php wp-load.php wp-settings.php xmlrpc.php
readme.html wp-blog-header.php wp-config-sample.php wp-includes wp-login.php wp-signup.php
[root@docker wp_vol]#
```

You're Done!

 You've successfully deployed a WordPress website using Docker with NFS for Data Persistence